OS Assignment-7
2401MC35

Report

1. Design Decisions and Key Data Structures

Question 1: Implementing User-Level Threads

- **Design Decision:** xv6 natively supports only full-weight processes via fork(), which duplicates the parent's entire page directory and memory space. To implement kernel-supported threads sharing a single address space, we introduced a clone(fcn, arg, stack) system call. Unlike fork(), clone() sets the child process's page table pointer (np->pgdir) to point directly to the parent's page directory (proc->pgdir), allowing both threads to share the identical virtual memory layout.
- **Key Data Structures:**
    - struct proc was extended with an int is_thread flag and a pointer to track the user stack (void *ustack).
    - The custom join() system call reclaims thread resources without destroying the shared page directory.

Question 2: Process/Thread Synchronization using Semaphores

- **Design Decision:** Counting semaphores were implemented in kernel space to prevent race conditions during concurrent variable updates and to manage the producer-consumer bounded buffer. We utilized xv6's native sleep() and wakeup() primitives paired with spinlocks to ensure atomic modifications of semaphore values.
- **Key Data Structures:**
    - struct semaphore { int value; struct spinlock lk; }
    - A fixed-size global kernel array containing 10 semaphores (struct semaphore semaphores[MAX_SEM]) protected by individual spinlocks.

Question 3: Deadlock Detection and Banker's Algorithm

- **Design Decision:** Because xv6 does not include a multi-resource kernel allocator, Questions 3 was structured as a comprehensive user-space simulation (deadlock.c). Static matrix allocations represent system states, and safety algorithms verify whether resource allocations can avoid circular wait conditions.
- **Key Data Structures:**
    - Multi-dimensional static matrices: Allocation[N][M], Max[N][M], Need[N][M], and 1D arrays Available[M] and Work[M].

Question 4: Priority-Based Preemptive CPU Scheduling with Aging

- **Design Decision:** The default xv6 Round-Robin scheduler was replaced with a priority-driven preemptive scheduler. Each process is assigned a priority value from 1 (highest) to 20 (lowest), defaulting to 10. The scheduler scans all RUNNABLE processes, selects the one with the lowest numerical priority value, and applies round-robin tie-breaking.
- **Key Data Structures:**
    - int priority and int wait_ticks added to struct proc to track scheduling priority and accumulated wait duration.

2. Problems Faced and How They Were Resolved

- **Page Faults on Shared Thread Exit:** Initially, when individual threads terminated, calling freevm() freed the shared page directory, causing subsequent memory faults in active sister threads. *Resolution:* The join() and exit handlers were modified to check the is_thread flag, ensuring the shared page directory (pgdir) is freed only when the parent process finally exits.
- **Race Conditions in Semaphores:** Early semaphore implementations experienced missed wakeups because checks on semaphore values occurred outside critical sections. *Resolution:* Every semaphore operation (sem_wait, sem_signal) was strictly bound to an internal kernel spinlock before evaluating block/wakeup conditions.
- **Process Starvation in Priority Scheduling:** Low-priority background tasks were starved indefinitely under heavy workloads. *Resolution:* An **aging mechanism** was introduced into the scheduler loop: any RUNNABLE process unselected for more than 30 ticks has its priority value decremented by 1 (boosting its priority), preventing indefinite starvation.

3. Before/After Comparison of the Race Condition (Q1 vs. Q2)

Metric / Aspect	Without Synchronization (Q1 - threadtest)	With Semaphores (Q2 - synctest)
Execution Mechanism	3 threads incrementing a shared global counter concurrently	3 threads protected by a mutual exclusion semaphore (mutex_sem)
Observed Final Value	Variable, typically $< 300,000$ due to interleaved atomic write collisions	Exactly $300,000$ (100,000 increments $\times$ 3 threads)
Behavioral Output	Race condition leads to data loss and non-deterministic results	Critical sections are fully protected; zero data loss or interference

4. Deadlock Detection Results Across Test Cases (Q3)

Test Case Scenario	Matrix Configuration Status	Detection / Avoidance Result
Case 1: Safe State	Initial allocation leaves sufficient slack for all processes to finish sequentially.	**No Deadlock Detected**; state verified as safe.

Case 2: Resource Request	Process 1 requests an incremental resource block [1, 0, 2].	**Approved** by Banker's Algorithm; system remains in a safe state.
Case 3: Unsafe / Deadlock	Cyclic wait injected where resource needs exceed available vectors.	**Deadlock Detected**; unfinishable processes identified and isolated.

5. Scheduling Behavior: Completion Time vs. Priority (Q4)

The priority-based preemptive scheduler with aging was evaluated across 4 parallel CPU-bound test processes.

Process PID	Initial Priority Level	Aging Boost Applied?	Relative Completion Order
PID 4	2 (High)	No (Finished immediately)	1st (Fastest)
PID 5	8 (Medium-High)	No	2nd
PID 6	15 (Medium-Low)	Moderate	3rd

PID 7	20 (Lowest)	Yes (Incrementally boosted via wait-ticks)	4th (Completed successfully without starvation)